Heliostat: Harnessing Ray Tracing Accelerators for Page Table Walks

Yuan Feng
University of California, Merced
Merced, California, USA
yfeng44@ucmerced.edu

Yuke Li
University of California, Merced
Merced, California, USA
yli304@ucmerced.edu

Jiwon Lee
Samsung Electronics
Seoul, Republic of Korea
jiwon24.lee@gmail.com

Won Woo Ro
Yonsei University
Seoul, Republic of Korea
wro@yonsei.ac.kr

Hyeran Jeon
University of California, Merced
Merced, California, USA
hjeon7@ucmerced.edu

Abstract

This paper introduces Heliostat, which enhances page translation bandwidth on GPUs by harnessing underutilized ray tracing accelerators (RTAs). While most existing studies focused on better utilizing the provided translation bandwidth, this paper introduces a new opportunity to fundamentally increase the translation bandwidth. Instead of overprovisioning the GPU memory management unit (GMMU), Heliostat repurposes the existing RTAs by leveraging the operational similarities between ray tracing and page table walks. Unlike earlier studies that utilized RTAs for certain workloads, Heliostat democratizes RTA for supporting any workloads by improving virtual memory performance. Heliostat+ optimizes Heliostat by handling predicted future address translations proactively. Heliostat outperforms baseline and two state-of-the-arts by 1.93×, 1.92×, and 1.66×. Heliostat+ further speeds up Heliostat by 1.23×. Compared to an overprovisioned comparable solution, Heliostat occupies only 1.53% of the area and consumes 5.8% of the power.

CCS Concepts

• **Computer systems organization → Single instruction, multiple data**; • **Computing methodologies → Graphics processors**.

Keywords

GPU, Virtual Memory, Address Translation

ACM Reference Format:
Yuan Feng, Yuke Li, Jiwon Lee, Won Woo Ro, and Hyeran Jeon. 2025. Heliostat: Harnessing Ray Tracing Accelerators for Page Table Walks. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3695053.3731011

1 Introduction

With the increasing size and complexity of GPU applications, virtual memory systems have become one of the critical performance hurdles in GPU computing. Various studies improved the performance through adaptive TLB coalescing [3, 21, 38], page mapping coalescing [12], multi-GPU TLB sharing [12, 25, 40, 41], and lazy invalidation [2, 20, 22]. These earlier studies mostly focused on reducing the latency of individual page translations or optimizing the provided GPU memory management unit (GMMU) bandwidth. Thus, their performance impact is limited to the bandwidth of the virtual memory system integrated in the GPU. To support larger and more diverse workloads, we should explore efficient ways to increase the translation bandwidth fundamentally.

Figure 1 shows the performance impact of increased translation bandwidth. When we integrate more page table walkers (PTWs) to the GMMU, most applications show dramatically improved instructions per cycle (IPC). The Low, Mid, and High in the graph indicate the translation intensity according to L2 TLB misses per kilo instructions (L2 TLB MPKI). Especially the workloads with high translation intensity exhibit significant speedup; more than half of the workloads derive over 3× speedup with 128 PTWs over the baseline 16 PTWs. This indicates that there is considerable potential to enhance performance by increasing the translation bandwidth.

However, adding more PTWs is not a scalable solution because the speedup is achieved at the cost of area and power. From our RTL-level evaluation, 128 PTWs incur over 7.8× area and 6.8× power overhead than the baseline 16 PTWs, as shown in Figure 1. Such overhead might be amortized if all applications could achieve notable speedup. However, almost all applications with low intensity and half with middle intensity barely benefited from more PTWs. For those applications, the overprovisioned hardware resources will only occupy the die area consuming static power.

To support diverse workloads without overprovisioning GMMU, we propose *Heliostat*, which expands the translation bandwidth by repurposing the underutilized existing hardware components. Besides general-purpose cores, modern GPUs embed various accelerators such as tensor cores and ray tracing accelerators. These accelerators use fixed-function units specially tailored for certain types of applications. Due to their limited functionality, the accelerators are significantly underutilized when the GPU isn't running the target applications. To increase the utilization of these accelerators, several studies proposed to democratize the accelerators for various other applications that have operational similarities [7, 14, 49, 58]. These solutions require substantial modifications in both software



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.
ISCA '25, Tokyo, Japan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1261-6/25/06
https://doi.org/10.1145/3695053.3731011

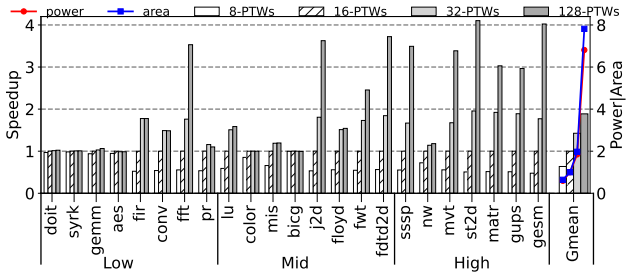


Figure 1: Normalized IPC, power and area with different number of PTWs

and hardware and the performance impact is still limited to the supported applications. On the other hand, Heliostat is a hardware-only solution, which expands the translation bandwidth by leveraging these underutilized accelerators. Thus, Heliostat can enhance the performance of *any* applications experiencing translation overhead.

We especially repurpose ray tracing accelerators (RTAs) by exploiting the operational similarity between ray tracing and page table walk. As the main function of ray tracing is to traverse the path of lights (rays) and check the intersected objects in the 3D scene, RTAs are equipped with hardware components that support tree traversals, such as ray buffers and dedicated memory units. As GMMUs use radix tree-based multi-level page tables, RTA's tree traversal capability fits well with the page table traversal.

However, there are challenges to repurposing RTAs for PTW operations: **First, RTA's pipeline and data structures are not compatible with page translation.** To support PTW operations, we define new data structures and design a translation unit with lightweight modifications in the RTA architecture. The translation unit is added as one of the operation units in the ray tracing pipeline. Hence, the unit is activated similar to the existing operation units. **Second, an RTA and other cores share a small L1 data cache.** Sharing a small data cache with other cores brings two concerns; cache pollution and loss of page table walk locality. To resolve this problem, we exploit another underutilized hardware component, which is the constant cache. We show the significant underutilization in the shared constant caches and propose to reserve some space in the constant cache to be exclusively used by the RTA page translations. **Third, there is no existing method to distribute translation requests between PTWs and RTAs.** To arbitrate the translation job assignment between the GMMU and RTAs, we design a new PTW forwarding unit. By utilizing the resources in the RTAs, Heliostat attains a speedup (1.93×) comparable to a GMMU with 128 PTWs, while using only 1.53% of the area and 5.8% of the power consumption of 128 PTWs.

Based on the design of Heliostat, **we further harness the existing resources to enable proactive *lookahead translations*** with namely *Heliostat+*. While Heliostat focuses on supporting on-demand translations, Heliostat+ enables translations for predicted future requests. Heliostat+ leverages the existing ray tracing function, *secondary ray*. In ray tracing, when a primary ray reaches a translucent surface, a secondary ray is forked and continues the traversal in the reflected direction with the traversal information inherited from the primary ray. By leveraging this existing capability, Heliostat+ handles a pair of an on-demand translation and a lookahead translation at the same time. Similar to the secondary ray, the lookahead translation thread is forked only when its page table traversal path diverges from the on-demand translation. This enables saving multiple shared page table traverses. The translation operations can be skipped when an on-demand translation can be served from the cached lookahead translation result. Heliostat+ achieves speedup over Heliostat by 1.23× (2.39× over the baseline) and superior performance than two state-of-the-art solutions by 1.93× and 1.66×.

Our contributions are as follows:

- To the best of our knowledge, this is the first study that leverages the underutilized RTAs to improve the performance of the GPU virtual memory system. Unlike existing RTA democratization studies that can support limited scope of applications, Heliostat accelerates general-purpose system operations.
- Unlike existing studies that primarily focused on optimizing the existing translation bandwidth, Heliostat demonstrates a lightweight method to fundamentally increase the translation bandwidth with significantly lower area and power overheads compared to the GMMU overprovisioning.
- Heliostat+ demonstrates a novel proactive lookahead translation without a notable memory traffic increase by applying a ray tracing feature that allows sharing the page table walks between on-demand translations and lookahead translations.

2 Background

2.1 GPU Architecture and Virtual Memory

Figure 2 illustrates the baseline GPU architecture. A GPU consists of multiple Compute Units (CUs), each containing register files, warp schedulers, compute cores, and memory execution units (MEUs). Each CU uses an L1 Vector (L1V) cache as a private data cache. All CUs in a Shader Array (SA) share an L1 Scalar (L1S) cache for storing constant data, such as kernel arguments and pointers [1, 6, 48]. Each CU is equipped with a Matrix Core for deep learning workloads and a Ray Tracing Accelerator (RTA) for real-time ray tracing rendering. To support the virtual memory system, each CU uses a private L1 Translation Lookaside Buffer (TLB) and all CUs share an L2 TLB. Upon an L2 TLB miss, a page translation request is sent to the GPU Memory Management Unit (GMMU) to be served by an available page table walker (PTW). Each PTW traverses over the radix-tree-based multi-level page tables. The GMMU incorporates 16 PTWs [8, 12, 16, 21, 22, 24, 40, 41, 46–48, 53].

2.2 Ray Tracing and Hardware Support

2.2.1 Ray Tracing. Ray tracing (RT) is a graphic rendering algorithm that simulates the lighting effects to generate highly realistic scenes. The RT algorithm tests the intersection between rays and the graphic primitives, e.g. triangles and spheres, in a scene. A large number of rays are traced concurrently to generate a high-quality scene. Modern RT libraries organize the graphic primitives into acceleration structures (AS) that segment the primitives in the scene. Bounding Volume Hierarchy (BVH) is the de facto standard AS that

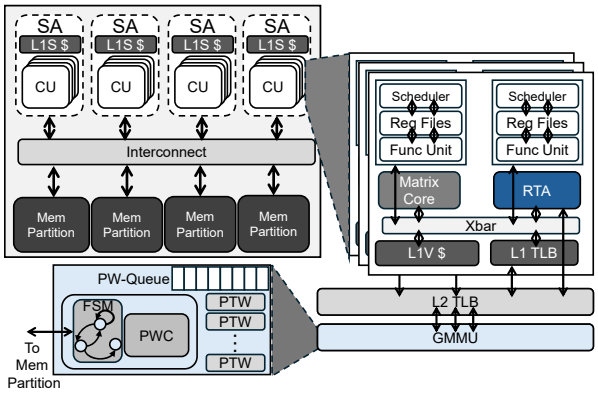
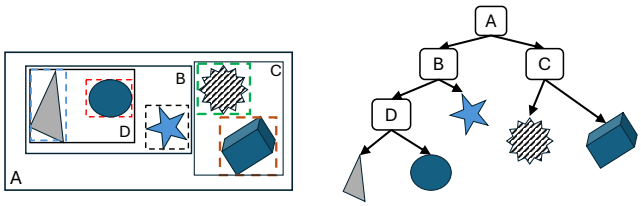


Figure 2: Baseline GPU Architecture



(a) 2D scene example **(b) BVH Tree example**

Figure 3: Examples on BVH Tree

has been used in many industry products including Vulkan [52], NVIDIA RTX GPU [35], and Intel Arc GPU [15].

2.2.2 BVH and BVH Traversal. Figure 3 shows an example 2D scene and the BVH built from the scene. In the scene, the graphic primitives are placed inside a graphic world A. The scene is then segmented into two sub-regions (B and C) by drawing the axis-aligned bounding boxes (AABBs) according to the location of the primitives. Such segmentation continues recursively until the leaf nodes contain the actual primitives. A BVH tree is built upon completion of finding all leaf nodes as shown in Figure 3b.

To find all and closet graphic primitives that are intersected with individual rays, ray tracing runs intersection testing functions while traversing BVH trees. BVH trees are typically traversed through depth-first search (DFS). Whenever an intersected primitive is found, a callback function *any-hit* share is called. When the traversal finishes, the *closest-hit* function is called back.

2.2.3 Ray Tracing Accelerator. Though GPUs are massively parallel processors, the lock-step nature of single-instruction-multiple-thread (SIMT) execution makes it inefficient to process tree structures like BVH trees. To enable real-time RT, GPU vendors integrate RT accelerators (RTAs) into GPUs, such as raytracing accelerators in AMD, RT Core in NVIDIA, and RT Unit in Intel Xe-cores. Figure 4 illustrates a typical RTA architecture. Once a warp is dispatched from a CU's issue scheduler, the warp is enqueued to the *Warp Buffer*. The ray information (*Ray Property*) is also enqueued to the warp's private *Ray Buffer*. An RTA can handle four concurrent warps [14, 43]. At every cycle, the warp scheduler in an RTA selects a warp from the Warp Buffer to traverse the input BVH tree. While traversing the tree, once a node is fetched via the dedicated memory unit (*Mem Access FIFO*), the node is passed to *Node Decoder*. Then,

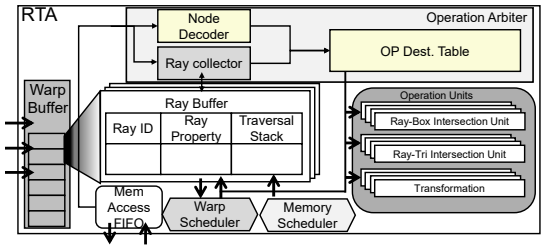


Figure 4: Baseline Architecture of Ray Tracing Accelerator

the Node Decoder decodes the node by using the ray properties in the Ray Buffer and identifies the next ray operation out of the three *Operation Units*. *OP Dest. Table* routes the execution to the selected ray operation unit. Once the ray operation is finished, the traversal stack and ray property are updated in the Ray Buffer.

3 Motivation

Methodology: We modeled an AMD GCN GPU by using a cycle-level GPU simulator, MGPUSim [48]. A GPU has 128 CUs. Each CU has one RTA. We used the RTA architecture model of Vulkan-sim v2.0 [43]. We employed 16 PTWs in the GMMU [8, 12, 16, 21, 22, 40, 48, 53]. L2 TLB is a 16-way set-associative cache with 512 lines. Detailed simulation configurations are shown in Table 2. We used 23 general-purpose GPU applications selected from widely used benchmark suites. We classified individual applications into Low, Mid, and High, according to their address translation intensity, which is calculated by L2 TLB Misses Per Kilo Instructions (MPKI), as shown in Table 1.

High L2 TLB misses in GPU computing: In Section 1, we showed a substantial speedup with increased translation bandwidth. To understand the reason behind the performance improvement, we measured the L2 TLB miss rates. The results in Figure 5 show that applications in the High group barely benefited from an L2 TLB; on average, every other TLB access encounters a miss. Although

Table 1: Tested Workloads and Benchmarks

Benchmark	Application	L2TLB MPKI	Abbr	Category
POLYBENCH	doitgen	0.015	doit	Low
POLYBENCH	syrk	0.017	syrk	Low
POLYBENCH	gemm	0.028	gemm	Low
HeteroMark	aes	0.071	aes	Low
HeteroMark	fir	0.319	fir	Low
AMD APP SDK	simpleconvolution	0.445	conv	Low
SHOC	fft	0.478	fft	Low
HeteroMark	pagerank	0.807	pr	Low
POLYBENCH	lu	1.27	lu	Mid
Pannotia	color	1.56	color	Mid
Pannotia	mis	2.08	mis	Mid
POLYBENCH	bicg	2.11	bicg	Mid
POLYBENCH	jacobi2d	2.16	j2d	Mid
AMD APP SDK	floydwarshall	2.24	floyd	Mid
AMD APP SDK	fastwalshtransform	2.27	fwt	Mid
POLYBENCH	fdtd2d	2.49	fdtd2d	Mid
Pannotia	sssp	4.06	sssp	Mid
RODINIA	nw	21.1	nw	High
POLYBENCH	mvt	48.3	mvt	High
SHOC	stencil2d	59.3	st2d	High
AMD APP SDK	matrixtranspose	196.2	matr	High
MAFIA	gups	557.4	gups	High
POLYBENCH	gesummv	4784.5	gesm	High

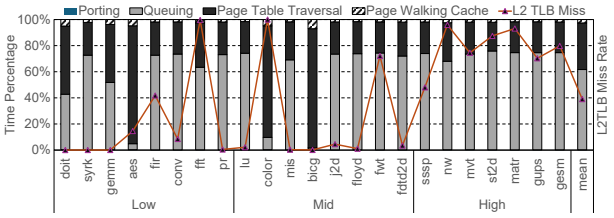


Figure 5: L2 TLB miss rate and miss panelty breakdown

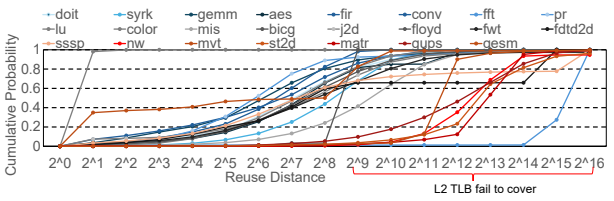


Figure 6: Cumulative distribution function (CDF) of translation reuse distances at the L2 TLB

the workloads in the Low and the Mid groups encounter lower miss rates than those in the High group, some applications such as `fft` and `color` show almost 100% L2 TLB misses even when the baseline L2 TLB is sufficiently large.

Can a larger L2 TLB help?: We then were curious if a larger L2 TLB could have more hits. To understand the proper TLB size of each application, we measured the translation reuse distances. Figure 6 shows the cumulative distribution function (CDF) of the L2 TLB reuse distances. Applications in each group are plotted in blue (Low), gray (Mid), and red (High). The workloads in the High group have significantly far reuse distances which begin from 2^9 up to 2^{16}. We also observed that `fft`, which is in the Low group but has 100% L2 TLB miss rate, has significantly far reuse distances beginning with 2^{14}. This means that these applications need a huge L2 TLB having several thousand entries to reuse any cached translations. Given that the most common L2 TLB setting [12, 21, 24, 25, 50] is to use 512 entries (as in our baseline), we should increase L2 TLB size by at least 10 times to reduce L2 TLB misses for these applications. Such a huge L2 TLB will lead to significant power and area overhead. More importantly, most of the applications in the Low and Mid groups will encounter significant underutilization because their reuse distances are shorter than 512. Therefore, increasing the TLB size is not a scalable solution.

Hint from miss penalty breakdown: The miss penalty statistics in Figure 5 also provide a hint about the main performance bottleneck. In most of the applications across all three translation intensity groups, the queueing delay contributes the most to the L2 TLB miss penalty. This means that most translation requests cannot be served quickly, mainly due to insufficient PTW bandwidth.

Based on these results, we can conclude that the limited GMMU bandwidth is the main performance bottleneck. Therefore, we propose to increase the translation bandwidth.

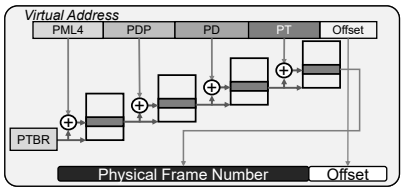


Figure 7: Multi-level Page Table Architecture

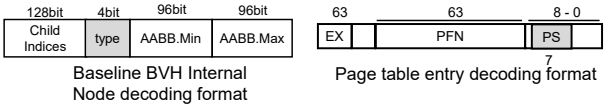


Figure 8: Formats of BVH tree nodes [29] and PTEs.

4 Address Translations with RTAs

4.1 Why RTAs for address translation?

To increase the translation bandwidth without significant overhead, we aim to utilize existing underutilized hardware components for translation. To substitute PTWs, the hardware component should 1) be able to issue memory transactions without interfering with regular memory operations, 2) be able to traverse tree-based multi-level page tables, and 3) be available when existing PTWs are fully occupied without delaying their own operations.

RTAs fulfill all three requirements. The RTA is the only architecture component besides the memory execution unit (MEU) (equivalent to LDST unit in NVIDIA GPUs) in the CU that can independently issue memory transactions. The RTA by nature supports tree traversal operations as described in Section 2. Although the page table architecture is different from the BVH tree, PTW operations use DFS, like in the BVH tree traversal, and the RTA is designed to accelerate DFS. Furthermore, RTAs are specialized cores that are used only for RT operations. Therefore, when a GPU runs general-purpose applications that barely use RT operations, RTAs are guaranteed to be idled throughout the execution. Therefore, we propose to leverage RTAs for address translation.

4.2 Challenges of Repurposing RTAs

There are three challenges to running PTW operations on RTAs.

First, the RTA pipeline and data formats are not compatible with PTW operations. (1) Tree format: The BVH tree is a full binary tree, while the page table is not, as shown in Figure 7.

(2) Traversal termination condition: Both RTA and PTW terminate the tree traversal when they finish processing a leaf node. In RT, the node type information is encoded in the tree node data structure, as shown in Figure 8. On the other hand, to indicate the leaf node, PTWs use two methods. For the default size pages (4KB), PTWs traverse for the statically determined page table depth e.g., in Figure 7, PTWs can retrieve the physical frame number (PFN) after four tree traversal steps. If the GPU uses large pages (e.g., 64KB or 2MB), the traversal should stop earlier because the tree depth is shorter. In this case, PTWs check a one-bit last-level table indicator (e.g., *PS* bit in Figure 8) in the page table entry (PTE). None of both methods are implemented in the RTA's *Node Decoder*.

(3) Next-level child node location: In RTA, the next-level node location is also encoded in the node data structure. On the other

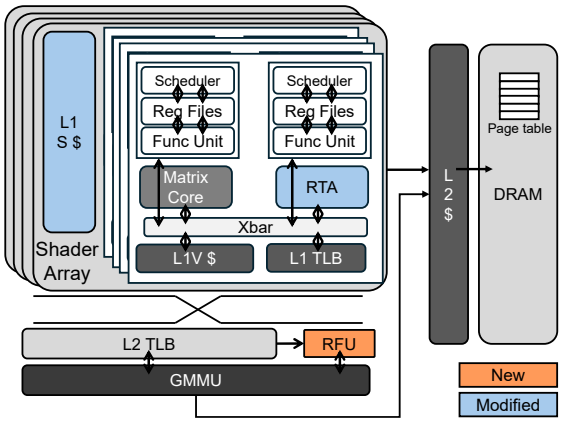


Figure 9: Modified Architecture with Heliostat

hand, PTW should calculate the location by using a subset of the bits in the requested virtual page number (VPN), as shown in Figure 7.

(4) Node format: As illustrated in Figure 8, the formats of a 40-byte BVH tree node and a 64-bit PTE do not have any similarity.

(5) Operations: The operation units in the RTA are variants of ray intersection testing functions. However, page translation does not share functional similarities with ray intersection testing. The address translation includes extracting a PFN, checking the access control, and triggering a page fault. None of these three functions can be natively executed by the RTA's operation units.

Second, an RTA shares L1V cache with regular memory operations. While an RTA does not use a MEU unit to issue memory transactions, caches are shared between MEUs and the RTA. Therefore, there is a potential risk of cache pollution, that may drop the performance of regular memory operations. On the other hand, page table walk operations are highly benefited from caches because a few upper-level page tables are frequently shared by many translations. To support such locality, GMMUs are typically equipped with page table caches. Therefore, RTAs should have a separate cache space. An RTA has internal buffers such as *Ray Buffers* but they are private resources for individual warps. Therefore, page table information cannot be shared across RTAs.

Third, there needs an arbitrator to distribute the translation tasks between the GMMU and RTAs. As the goal of Heliostat is to expand the translation bandwidth, the translation requests should be assigned based on the availability of the GMMU and RTAs. However, existing GPU architectures do not have such arbitrators.

5 Heliostat

5.1 Overall Architecture

Figure 9 shows the updated GPU architecture with Heliostat. We modify RTAs to translate addresses upon L2 TLB misses (Section 5.2). Heliostat triggers a page translation on an RTA when there is no available PTW. To check the status of PTWs in the GMMU and redirect a translation request to an RTA, we design an *RT-PTW Forwarding Unit (RFU)* (Section 5.3). An RFU is shared by all CUs in a GPU. To mitigate potential L1V cache pollution and leverage translation locality, we reserve space in the highly underutilized L1S cache for the RTAs (Section 5.4). The following sections describe the details of each of these components.

5.2 RTA Modification

Figure 10 shows the updated RTA architecture. We introduce a new operation unit, *PTE decoding unit (PDU)*, to traverse the multi-level page table. The PDU is responsible for handling the computations of PTW operations by utilizing the existing Ray Buffers and Mem Access FIFO in the RTA.

5.2.1 Ray PTW Property. Ray property is used for tracking the traversal status. Figure 11 shows the original *RayProperty* data structure used by the RT and our proposed *RayPTWProperty* to be used for PTW operations. The original ray properties are mostly used for describing a ray (e.g., the origin and the direction of the ray). Our proposed structure incorporates information for PTW; `V_addr` (64 bits) contains the VPN to be translated; `pid` (32 bits) stores the process ID; `rwx` (8 bits) stores the access control information; `level` (8 bits) is used to carry the current page table walk level; `P_addr` contains the translated physical address.

When a translation request is received, a warp is scheduled and a new *RayPTWProperty* instance is created for the warp. The *RayPTWProperty* is maintained in the Ray Buffer and updated at every page table traversal step. When the translation finishes, the Mem Access FIFO in the RTA responds to the RFU with the translated `P_addr`. Compared to the original *RayProperty* (256 bits), the proposed *RayPTWProperty* (112 bits) is smaller. Therefore, we can use the proposed data structure without extra storage space.

5.2.2 Node Decoder. The Node Decoder in an RTA is used for decoding the node type (e.g., leaf or internal node) and activating a proper operation unit. In the page translation, as discussed in Section 4.2, when the GPU uses 4KB pages only, we simply traverse for the statically determined tree depth. Therefore, we do not need to decode any input. If the GPU uses diverse page sizes, we should check the *PS* bit in the PTE. To support both methods, we add a 1-bit checker in the Node Decoder, which checks the *PS* bit from a given PTE input. The bit value is passed to the PDU as an input. Upon receiving a translation request from RFU, Node Decoder makes the OP Dest. Table to activate the PDU with the *PS* bit value.

5.2.3 OP Dest. Table. The main function of the OP Dest. Table is to activate the correct operation units upon receiving the Node Decoder output. To activate the proposed PDU for the translation operations, OP Dest. Table is revised to integrate one more output port to pass the inputs to the PDU out of the operation units.

5.2.4 PTE Decoding Unit (PDU). The PDU handles the computation part of the PTW operations. Figure 12(a) shows the architecture. A PTW has three functions: 1) checking if the PTE is from the last-level page table, 2) if the PTE is from an intermediate-level page table, retrieving the base address of the next-level page table from the input PTE and issuing a memory transaction to fetch the next-level PTE, and 3) if the PTE is from the last-level page table, retrieving the PFN from the PTE and checking the access control and the valid bit to finish the translation. The following describes how each function is implemented.

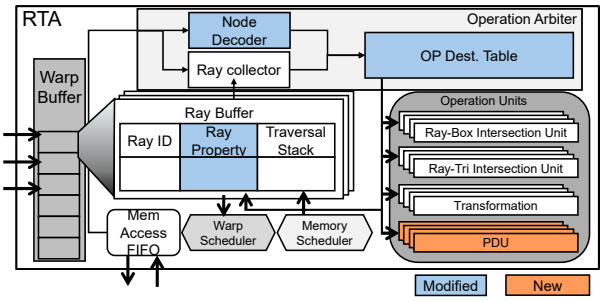


Figure 10: RTA with PTW data path

```
struct RayProperty
{
    /*!< Ray origin */
    rtFloat3 origin;
    /*!< Ray time for motion blur */
    float time;
    /*!< Ray direction */
    rtFloat3 direction;
    /*!< Ray maximum distance */
    float maxT;
};
```

(a) Baseline Ray Property field in Ray Warp Buffer

```
struct RayPTWProperty
{
    /*!< Virtual Addr */
    uint64_t V_addr;
    /*!< PID of requester process*/
    int32 pid;
    /*!< Read/write/execution */
    uint8_t rwx;
    /*!< Current PT walk level */
    uint8_t level;
   /*!< Physical Addr */
    uint64_t P_addr;
};
```

(b) Proposed Ray Property for PTW

Figure 11: Ray Property for PTW

(1) Checking the last-level PTE: The PDU first checks the *PS* bit input passed by the Node Decoder. If it is 1, the PDU determines that the current input is a last-level PTE for a large page. Otherwise, the PDU checks the `level` parameter of *RayPTWProperty*. The `level` is initialized as zero and incremented by one as the page table tree is traversed further. Thus, PDU checks if `level` value matches the system's page table depth. If matched, the PDU determines that the input PTE is from the last-level page table. To implement this function, we add two comparators, one for checking the input *PS* bit and another for checking the 8-bit `level` value.

(2) Calculating the next-level page table address: This function is implemented by taking the corresponding level's index value from the input VPN (e.g., one of `PML4`, `PDP`, `PD`, `PT` in Figure 7), multiplying the index with the size of a PTE, and adding it with the next-level page table base address which is retrieved from the input PTE. For the first-level page table, the *page table base register (PTBR)* is used for the base address. To implement this, we add one adder and one shifter. The calculated next-level page table address is passed to the Mem Access FIFO to issue a memory transaction. Then, the `RayPTWProperty.level` is incremented by one and passed to the Ray Property field in the Ray Buffer.

(3) Retrieving the PFN and checking access control: This is the final step of the PTW. The input PTE has the PFN. The PFN is extracted and updated to the `RayPTWProperty.P_addr` field when the PTE's valid bit is set and the access control field values match with `RayPTWProperty.rwx`. If access control values mismatch or the valid bit is not set, the PDU returns the translation request to the RFU so that the GMMU can replay the translation i.e., handling the page fault. To implement this, we add two comparators, one for checking the valid bit and another for checking the 8-bit `rwx` value.

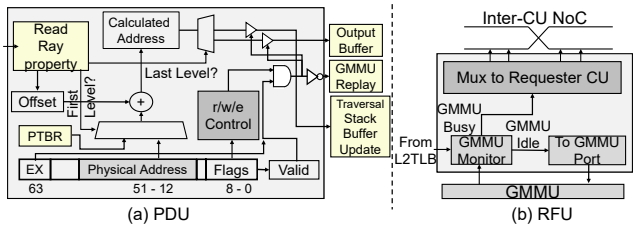


Figure 12: PTE Decoding Unit (PDU) and RT-PTW Forwarding Unit (RFU)

5.3 RT-PTW Forwarding Unit

We design an RFU to arbitrate the translation requests. The RFU determines whether the new translation request should be handled by an RTA or by the GMMU by monitoring the utilization of PTWs in the GMMU. Figure 12(b) shows the RFU architecture. The *GMMU monitor* probes the available PTWs. If there are any available PTWs, the incoming requests are sent to the GMMU. Otherwise, the requests are directed to the requesting CU's RTA via the inter-CU interconnection network.

5.4 Locality-aware Page Table Sharing

To avoid L1V cache pollution and also incorporate an exclusive page table cache, we propose reserving a space in a shared L1S cache for the RTA. Unlike an L1V cache, which is the main data cache for individual CUs, an L1S cache is a read-only cache shared by all CUs in each Shader Array. As an L1S cache is used for storing kernel arguments and data allocated with a constant qualifier [1, 6, 48], an L1S cache is typically highly underutilized. According to our evaluation, the L1S cache utilization is from 1% up to 19%. Also, as the L1S cache is shared by multiple CUs, the page table walk can be shared across RTAs. Thus, we leverage some of the unused L1S space as the page table cache, namely *RT-PTW cache*.

We can reserve space in a cache in several granularities e.g., one whole set, a subset of ways within each set, or a subset of words within each cache line [9, 10, 44, 56]. The way-level reservation has the least design complexity because it doesn't need to change the regular cache operations. Once a set is indexed by the regular set-associative policy, we can skip checking or updating the reserved ways out of all ways in the set. The set- or word-level reservations require modifications on the set associativity and cache line layout. Thus, we chose to design the way-level reservation. To reduce performance impact for applications that might use the L1S cache actively, we reserve only one way per set for the RTA operations.

Figure 13 shows the modified L1S cache. We add an *access mask register* to the cache controller. The register is a bit vector that has as many bits as the number of ways per set in the cache. The initial value of the register is all 1's. When we want to use a specific way exclusively for RTAs, we unset the corresponding bit. In Figure 13, way 2 is used for the RTA. The regular operations use the ways with corresponding bits set.

5.5 Operational Example

Figure 14 illustrates an end-to-end walkthrough of Heliostat. Suppose a translation request misses from both L1 TLB ① and L2 TLB ②. Then, the request is sent to the proposed RFU ③. If the GMMU has

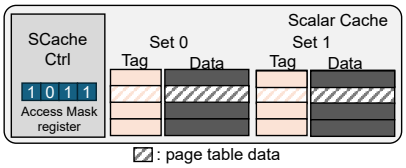


Figure 13: L1S Cache Reservation for RTA

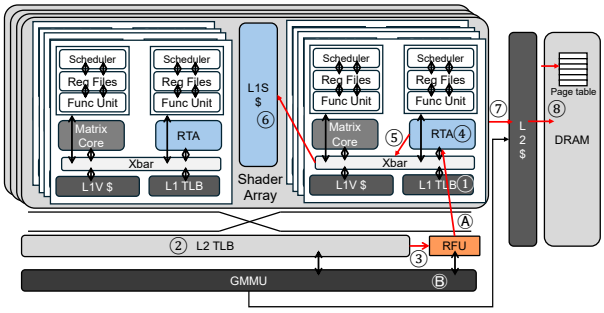


Figure 14: Heliostat Walkthrough

an available PTW, the request is sent and served by the GMMU Ⓑ. If the GMMU is busy, the translation request is sent to the requester CU's RTA Ⓐ, and our proposed PTW-enabled RTA will serve the translation request ⑤. During the translation, the RTA looks up the RT-PTW cache reserved in the L1S cache with the next-level page table address ⑥. If the page table is not cached ⑦, a memory transaction is issued to fetch the page table from the memory ⑧. Once the translation is finished, the translation results are responded to the RFU and then to the L2 TLB and the L1 TLB.

6 Heliostat+

6.1 Overview

Heliostat supports on-demand translations. However, we have not yet fully unleashed the potential of the RTA's functionality. Besides BVH tree traversal, RTAs also support handling *secondary ray*, which is a built-in functionality in ray tracing [42, 43, 45]. As shown in Figure 15, when a ray intersects with a reflective or translucent surface (e.g., glass), either ray tracing software or RTA creates another ray query that originates from the reflection location and travels in another direction to render reflection and transparency. By leveraging this existing functionality, we propose to support *lookahead translation*.

The lookahead translation is a proactive translation for the predicted future requests. Once translation requests follow a predictable pattern, Heliostat+ invokes a lookahead translation thread (secondary ray) to handle the predicted future translations (Section 6.2). Similar to the secondary ray, the lookahead translation thread begins the operation when its traversal path diverges from the on-demand translation thread's (primary ray) by inheriting the translation information. Therefore, the lookahead translation fits naturally with the existing secondary ray functionality. The lookahead translation results are stored in the L1S cache (Section 6.3). Once a translation request hits in the L1S, the stored translation is used and the translation operations can be skipped. The RFU is revised to detect strides and find the most probable RTA that might

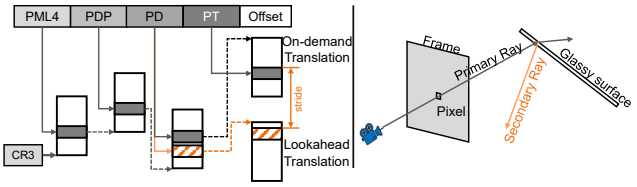


Figure 15: Lookahead Translation (left) and Secondary Ray Tracing (right)

have the cached translation by monitoring the translation requests (Section 6.4). When the RFU sends a translation request to an RTA, it adds a lookahead translation request in the same request packet by encoding the stride value in the packet header.

6.2 Lookahead Translation in RTA

The secondary ray can be handled by either software or hardware. We conservatively assume that the RTA does not have a built-in logic for the secondary ray and propose a lightweight modification in the RTA to support hardware-oriented lookahead translation.

6.2.1 Lookahead Translation Thread. The secondary ray inherits the most recent traversal information of the primary ray. To enable information sharing, the secondary ray is assigned to a thread of the warp of the primary ray. When a secondary ray is created, the Ray Buffer entry of the primary ray is copied to a new entry in the same Ray Buffer. The secondary ray thread begins the execution using the new entry. Similarly, we modify the RTA to create a new row in the Ray Buffer when a lookahead translation thread is created.

6.2.2 Ray PTW Plus Property. To accommodate the lookahead translation information and indicate the thread used for the lookahead translation, we added two new parameters in the *RayPTW-Property*. A 64-bit `Second_V_Addr` stores the lookahead translation address. The lookahead address is calculated by adding the stride to the on-demand translation address. A 1-bit `is_prefetching` indicates the lookahead thread property.

6.2.3 Invoking Lookahead Translation. If the stride field in the translation request packet is not zero, RTA prepares the lookahead translation. The on-demand translation and the lookahead translation might share a few upper level PTEs, as shown in Figure 15. To save memory transactions for the shared traversals, we invoke the lookahead translation only when the lookahead translation should access a different PTE from the on-demand translations. In Figure 15, the first two levels can be handled by the on-demand translation. Only from the third-level page table, the lookahead translation thread begins its traversal separately.

To support this, we modify the PDU and name it PDU+, as shown in Figure 16. The modifications are highlighted with orange color. We add a comparator to check if the lookahead and the on-demand traversals need to access different PTEs in the next-level page table. If so, we invoke a lookahead translation thread. The invocation signal is sent to the warp scheduler within the RTA and the scheduler assigns a new thread, creates a Ray Buffer entry for the thread, and schedules the lookahead translation.

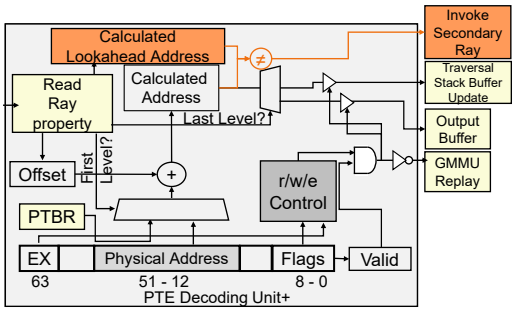


Figure 16: PDU+ with Lookahead Translation

6.3 Lookahead Translation Buffer

Unlike the on-demand translation, we do not update TLBs with the lookahead translation to avoid pollution. Given that the limited TLB reach is a pervasive performance problem [12, 13, 21, 25, 37, 38, 50, 55], we propose to maintain the lookahead translation results in a separate storage. Unlike earlier solutions that used a separate prefetch buffer [17, 33, 57, 60], Heliostat+ utilizes the L1S space that is reserved for Heliostat to minimize the area overhead.

While the L1S cache is used by Heliostat to maintain the cache lines loaded during the page table walks, the format and the size of a cache line (64 bytes) and a PTE (8 bytes) are different. Thus, naive mappings will waste the limited reserved cache space. If we store one PTE per cache line, 56 bytes will be wasted per cache line. If we store eight PTEs per cache line, we will need to store eight tags per cache line, which will complicate the design unnecessarily.

To resolve the mapping problem, we collocate eight PTEs having consecutive VPNs with a fixed stride. As Heliostat+ uses a simple stride-based lookahead translation prediction (Section 6.4), the PTEs tend to be distanced with an integer multiple of the stride size. Figure 17 shows the proposed mapping. Out of the reserved cache ways for Heliostat, we spare half of the cache lines as a lookahead translation buffer. The lines in the lookahead translation buffer are marked with a one-bit RT-PF indicator in the tags. Out of the eight PTEs having fixed strided VPNs, the smallest VPN is used as a tag for the cache line and filled in the *Base VPN* in the tag array. The stride can be any number but in our current design, we consider four common strides, which are 1, 64, 128, and 256 (Section 6.4). To indicate one of the four strides, we spare two bits in the tag.

The lookahead translation buffer is searched upon receiving the on-demand translation request. When there is a hit, we return the PTE to the RFU and skip the RTA translation. If the lookup misses, RTA begins the translation. When updating and searching the lookahead translation buffer with a VPN_A, we try $VPN_A \pm 8 \times$ stride. The lookahead buffer has only 64 cache lines. Therefore, the lookup does not incur much overhead.

6.4 Lookahead Translation Prediction

The proposed lookahead translation can support any translation prefetch algorithms as a separate thread handles the lookahead translation. In the current Heliostat+, we implement a simple stride-based algorithm to reduce the design complexity while demonstrating the efficiency of the RTA-based lookahead translation.

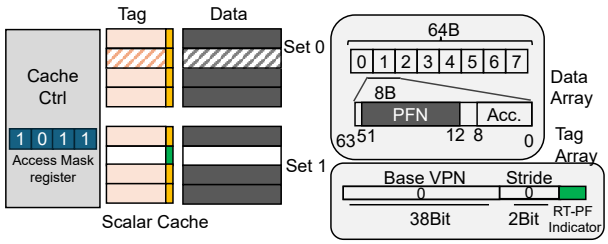


Figure 17: L1S Cache as a Lookahead Translation Buffer

6.4.1 Stride Detection. In GPUs, due to the massive parallelism, translation requests can be highly mixed up. To reduce such interferences, the RFU monitors a predetermined number (say R) of on-demand translations and calculates the candidate strides within each similar VPN region. For this, we implement a circular queue with R entries. We used 200 entries in our evaluation.

From our preliminary results, we commonly observed four strides: 1, 64, 128, and 256 pages. We believe this is mainly due to the number of CUs in the GPU (128 CUs in our baseline) and the common page mapping policies in GPU computing. Stride 1 indicates each CU accesses a few consecutive pages. Strides 64 and 128 mean that one page is shared by every two CUs and one CU, respectively. Thus, the current Heliostat+ is designed to choose one of the four strides. The stride is encoded in the translation request packet by leveraging the unused bits in address translation service packets [36].

6.4.2 Lookahead Translation-aware On-demand Translation Assignment. Depending on the page mapping policy, the lookahead translation results might be used by any CUs in the GPU. To share the lookahead translation results globally, we propose to predict the RTA that may have the cached translation already and send the request to the selected RTA. For this purpose, we add an *RT-PTW lookahead filter (RPF)* into the RFU. Out of various prediction algorithms, we use *cuckoo filter* [11]. A cuckoo filter is a probabilistic data structure that can predict whether a data item is possibly in the buffer or definitely not in the buffer. Unlike a similar bloom filter, a cuckoo filter supports efficient deletion operations.

We integrate as many cuckoo filters as the number of Shader Arrays. When an RTA finishes a lookahead translation, it sends a *filter-filling* message to the RFU. The RFU then inserts the VPN into the filter associated with the Shader Array that the RTA resides in. If a lookahead buffer evicts an entry, a *filter-deletion* message is sent to the RFU. As each lookahead buffer entry has eight PTEs, eight VPNs are deleted from the filter. Once RFU receives a translation request, it checks all filters. If any filter returns a hit, the translation request is sent to one of the CUs in the predicted Shader Array.

6.5 Operational Example

Figure 19(a) shows the walkthrough of triggering a lookahead translation. Suppose a translation request misses from L1/L2 TLBs ①②, the translation request is sent to the RFU+. If the RFU+ has a detected stride for the the requested address, the stride value is encoded to the on-demand translation request packet. Note that a lookahead translation is always accompanied by an on-demand translation, to take advantage of secondary ray. If the GMMU is available, only the on-demand translation will be handled by the

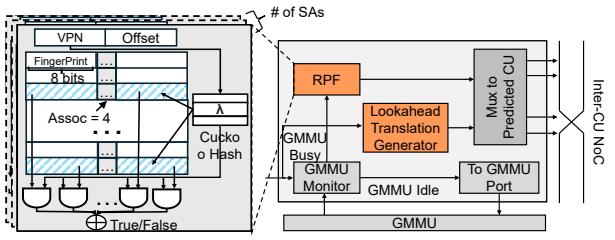


Figure 18: RFU+ with Lookahead Translation Prediction

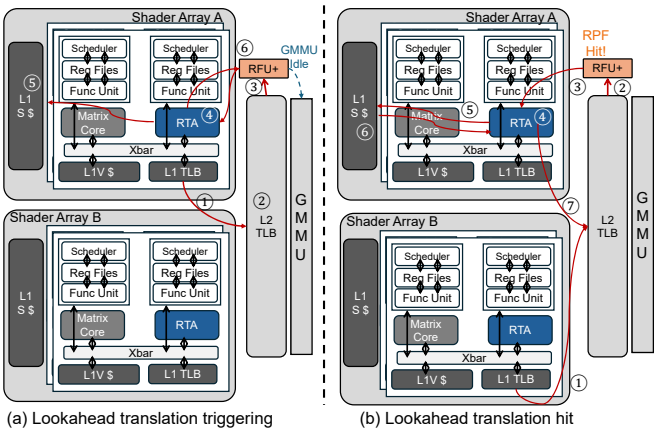


Figure 19: Lookahead Translation Walkthrough

GMMU since GMMU does not have the lookahead translation capability. The GMMU will ignore the stride bits. If the GMMU is busy, the on-demand request is sent together with the lookahead translation request ③. The lookahead request is served by an RTA ④, and the translated lookahead PTE is stored in the L1S ⑤. After the lookahead PTE is stored, a filter-filling message is sent to the RFU+, and the RPF for the Shader Array is updated.

Figure 19(b) illustrates how the lookahead translation is used for future translation. Suppose a subsequent translation request is made by a CU in a remote Shader Array B. If the request misses from both L1/L2 TLBs ①②, the request is sent to the RFU+ ②. The RFU+ then checks the RPFs and has a hit with the filter for Shader Array A ③. After that, the translation request is sent to the RTA in the Shader Array A ④. The RTA checks the lookahead buffer in the L1S ⑤. If the PTE is found ⑥, the RTA returns the PTE to the RFU+ without doing translations ⑦.

7 Discussions

RT operations with Heliostat-enabled RTA: Heliostat does not change any data path used by RT operations. When an RT kernel is invoked, Heliostat is not activated to avoid potential performance degradation. This can be controlled either by the programmer through a driver option or by the hardware through checking the execution of RT APIs. Technically, Heliostat and RT can be handled by an RTA concurrently because Heliostat uses dedicated operation units, PDUs. Unless RT workloads need the full capacity of warp buffers in all RTAs, Heliostat can be activated. To monitor the RTA utilization, we could add an RTA monitor similar to the GMMU monitor in the RFU. Then, upon TLB miss, RFU locates an available (or underutilized) RTA and sends the translation requests.

Security on PTBR added to the PDU: Heliostat adds PTBR in the PDU, which contains the top-level page table address. PTBR might be leveraged for page table leakage. Such leakage is not possible in Heliostat. The PDU is only accessed by hardware address translation requests, and only when the GMMU is busy. Therefore, TLB timing attacks to leak PTBR will be challenging to design.

Applicability to broader GPU models: Heliostat is based on AMD GPUs. But, NVIDIA GPUs feature similar RT Cores and virtual memory systems. Similar to shared L1S cache, NVIDIA Hopper's [34] distributed shared memory enables L1 data cache sharing among SMs within graphic processing clusters. Thus, RT-PTW cache and lookahead buffer can be implemented on NVIDIA GPUs.

TLB/Lookahead buffer coherence: Page faults or migrations trigger page table updates, creating stale PTEs in TLB. To maintain lookahead buffer coherent with TLB, Heliostat+ flushes lookahead buffer when system commands TLB flush upon page faults or page migrations. Since both GMMU and RTAs traverse the same page table copy, coherence between them is inherently maintained.

Multi-programming: Heliostat+ maintains seamless support for multi-programming by using *PID* for process differentiation. Testing with application combinations across categories demonstrates a 1.47× speedup over the baseline.

RPF false positive: RPF in Heliostat+ are probabilistic filters. It may cause false positive predictions. In this case, requests are handled as regular translation requests upon a miss in the lookahead buffer. Our evaluation shows a minimal false positive rate of 0.38%.

8 Evaluation

We evaluate Heliostat and Heliostat+ with the workloads and the simulation configurations as described in Section 3. The GMMU uses a conventional 4-level radix-tree-based page table and 16 PTWs. The PTWs share a 128-entry page walk cache.

8.1 Overall Performance

We compare our approach against a baseline using 16 PTWs without RTA, and two state-of-the-art solutions [8, 12]. Valkyrie [8] enables PTE and prefetched translation sharing between an L1 TLB and an L2 TLB upon detecting similar translation patterns, using lightweight NoC that connects L1 TLBs within Shader Arrays. Our Valkyrie implementation includes inter-L1 TLB communications and a 1024-entry prefetch history table in L2 TLB. BarreChord [12], focusing on GPU page table walk coalescing, allocates temporally similar page accesses to identical relative physical locations across GPU chiplets, enabling translation without page table walks. We implemented it by mapping GPU chiplets to individual Shader Arrays with cross-Shader Array page coalescing.

In Figure 20, Heliostat achieves 1.93×, 1.92×, and 1.66× speedups over the baseline, Valkyrie, and BarreChord respectively. For large applications, CUs within Shader Arrays rarely share L1 TLB entries due to exclusive page access patterns, making performance PTW-throughput bound and limiting Valkyrie's L1 TLB sharing benefits. While Barre's coalescing outperforms Valkyrie, it remains constrained by Shader Array boundaries. Heliostat achieves significant speedups through RTA offloading of PTWs, while Heliostat+

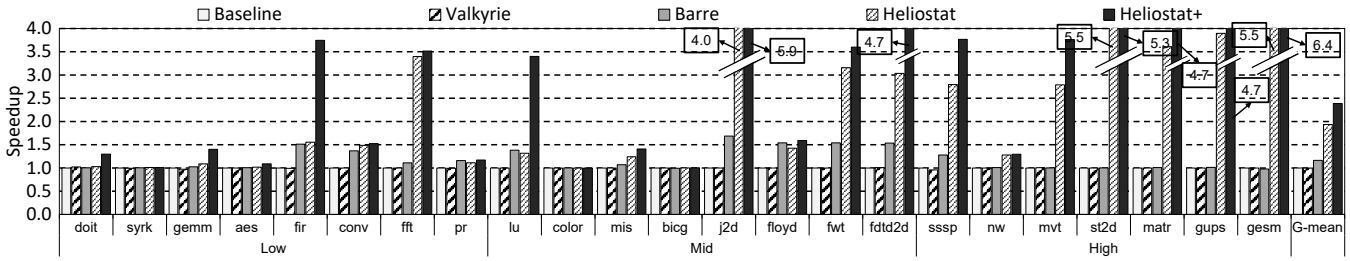


Figure 20: Overall Performance Comparison

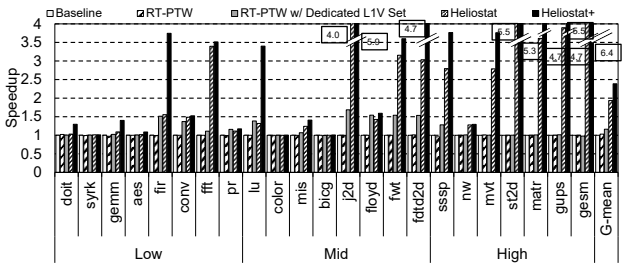


Figure 21: Performance Breakdown

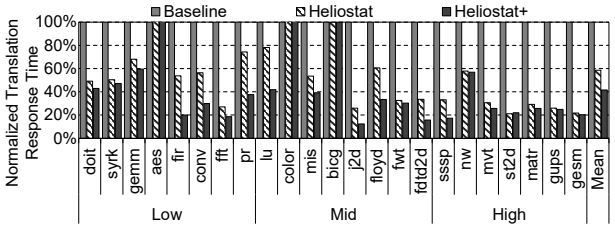


Figure 22: Normalized Page Translation Latency

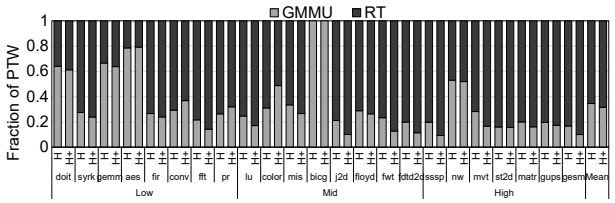


Figure 23: Fraction of translations served by RTAs and GMMU: in each application, H is Heliostat, H+ is Heliostat+.

further improves performance by 1.23× (2.39× over baseline) by improving both PTW bandwidth and latency through lookahead translations.

8.2 Performance Breakdown

Figure 21 shows performance across three optimizations. *RT-PTW*, which shares L1V cache with regular operations, achieves only 3% speedup over baseline, with some applications like `sssp` showing 4.3% degradation due to cache thrashing. *RT-PTW w/ dedicated L1V set* yields merely 1.16% improvement, as reduced cache capacity for regular operations impacts performance. Heliostat achieves 1.93× speedup by using shared L1S cache, allowing full L1V utilization for regular operations while providing dedicated, CU-shared cache space for RTAs. Heliostat+ further improves performance by 1.23× through lookahead translation with dedicated L1S buffers.

Table 2: Simulation Parameters

Parameter	Value
Number of CUs	128
L1 Vector Cache	64 KB, 4-Way, 32 MSHRs, LRU
L1 Inst Cache	64 KB, 4-Way, 4 MSHRs, LRU
L1 Scalar Cache	16 KB, 4-Way, 4 MSHRs, LRU
L2 Cache	2 MB, 16-way, 192 MSHRs, LRU
Network On Chip (NoC)	single-stage crossbar[8, 12, 24, 25, 53], ≈50-cycle latency
Warp Scheduler	GTO
DRAM	512 GBps
L1 TLB	64 entries, fully connected, 16 MSHRs, 1 cycle lookup latency, private to CU, LRU.
L2 TLB	512 entries, 16-Way, GPU shared [8, 12, 21, 22, 24, 25]. 10 cycle lookup latency, 128 MSHRs, LRU.
Number of RTA	1 per CU
Operational Unit	4 sets [43]
RTA MSHR Size	64 [43]
RTA Max Warp	4 [43]
GMMU	16 shared PTWs. 128 PW-queue entries [12, 40]
PW Cache	128-entry shared between PTWs [21, 24, 25, 40]

8.3 Page Translation Latency

In Figure 22, we present the normalized latency of page table walks. The latency includes the memory (cache) access latencies to load page tables, the time for PTE decoding, and the queueing delay until acquiring a PTW or an RT warp. With Heliostat, the translation latency is reduced by 41.5% on average. This is because Heliostat expands the PTW bandwidth with 128 RTAs, which reduces the queueing delay. Heliostat+ further reduces the translation latency by 16.9% because the lookahead translations eliminate the entire on-demand PTW latency when the lookahead buffer hits. The additional memory requests (7% and 0.6%) brought by lookhead translation in Heliostat+ has minor performance impacts.

8.4 Translation Workload Distribution

To understand how much translations are offloaded to RTAs, we measured the percentage of translation requests served by the GMMU and the RTAs. As shown in Figure 23, over 65.3% and 68.5% of translation are offloaded to the RTAs by Heliostat and Heliostat+ on average, respectively. This reflects that GMMU PTWs are supposed to be severely overloaded without our solution. Heliostat+ offloads more than Heliostat because Heliostat+ can skip some translations when lookahead buffer hits and serve more requests.

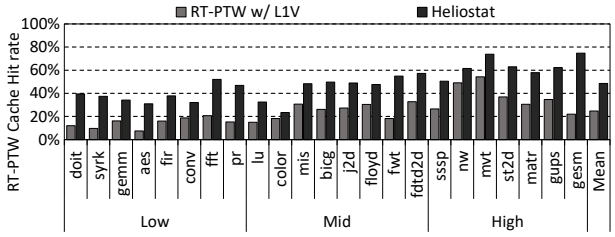


Figure 24: Hit rate of RT-PTW cache with L1V vs. L1S

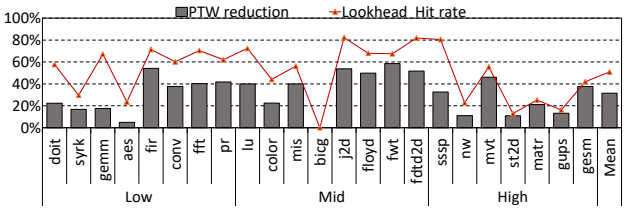


Figure 25: PTW reduction and prefetch hit rate

8.5 RT-PTW Cache Hit Rate

To evaluate the effectiveness of our locality-aware page table sharing, Figure 24 compares the cache hit rates between two designs: the baseline design that utilizes the general-purpose data cache (L1V cache) and Heliostat that reserves a way in the L1S cache as a dedicated RT-PTW cache. The baseline design achieves only a 24.6% hit rate, primarily due to the interference with the general data transactions. The proposed L1S-based RT-PTW cache achieves a 48.6% hit rate. This significant improvement demonstrates that Heliostat effectively reduces expensive memory transactions during the page table traversal.

8.6 Lookahead Buffer Hit Rate & Translations Reduction

In Figure 25, we present the lookahead buffer hit rate and the corresponding page table walk reduction. On average, 50% of the lookahead buffer entries encounter hits before eviction, and the hit entries reduce 31% of translation requests. Especially for the applications having regular access patterns such as `fir`, `fft`, and `j2d`, Heliostat+ achieves higher page table walk reduction and prefetching hit rate because those applications tend to follow relative uniform strides.

8.7 Sensitivities

8.7.1 Page Size. Heliostat supports multiple page sizes beyond the 4KB baseline. Figure 26 (left) shows Heliostat+ achieving 7% and 0.3% speedup with 64KB and 2MB pages respectively, despite large pages already reducing PTW loads through increased TLB reach and address coalescing. To assess scalability, similar to [12, 24, 41], we tested with 16× larger applications (which reflects the change from 4KB to 64KB). Figure 26 (right) shows results from a representative subset across translation intensities, with Heliostat+ achieving 31% and 4.2% speedup for 64KB and 2MB pages respectively.

8.7.2 Number of PTWs. Figure 27 shows Heliostat+'s speedup across different baseline PTW counts. With 32 PTWs, Heliostat+ achieves 1.32× speedup, while with 8 PTWs, it reaches 3.28× improvement, demonstrating particular effectiveness in resource-limited systems.

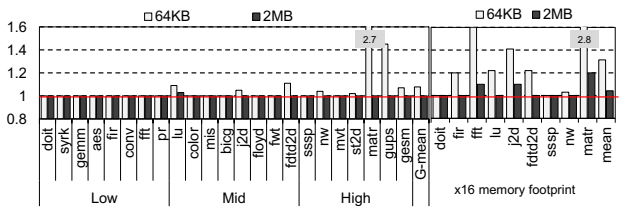


Figure 26: Speedup under Larger Pages

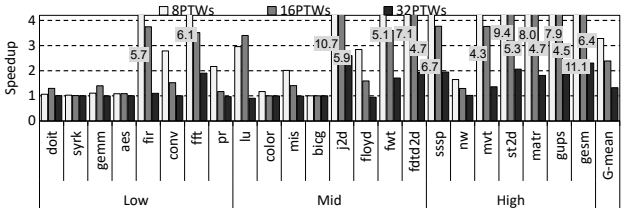


Figure 27: Speedup under different number of PTWs

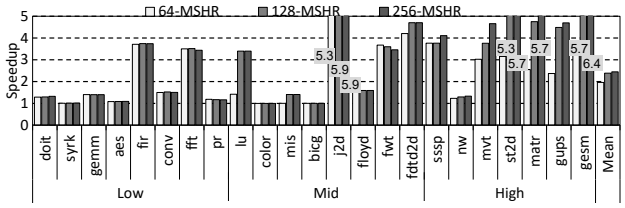


Figure 28: Speedup under different number of L2 TLB MSHRs

8.7.3 Number of L2 TLB MSHR. The number of MSHRs impacts the maximum concurrent translation requests. We provide a sensitivity analysis on the number of L2 TLB MSHRs in Figure 28. When there are half as many MSHRs, the speedup decreases to 1.96×; this is because the concurrency is capped by the MSHR number. When the number of MSHRs doubles, Heliostat+ is able to further utilize the RTAs' parallelism and therefore the speedup increases to 2.44×.

8.7.4 NoC Latency. Translation requests are delivered to RTA via NoC. To understand the impact of NoC latency, we evaluated the speedup of Heliostat+ when the NoC latency is increased by 1.5 and 2 times. As shown in Figure 29, even with the significant latency increase, Heliostat+ achieves notable speedup over the baseline (2.32× and 1.90×, respectively). Interestingly, some applications in the `high` category show even higher speedup with higher NoC latency. This is because the increased NoC latency slows down the PTW request traffic as well. In other words, Heliostat+ can derive even better speedup when the NoC is slower.

8.7.5 TLB Latency. While we used the common TLB configurations used by many studies [12, 22–25, 53], we evaluate the impact

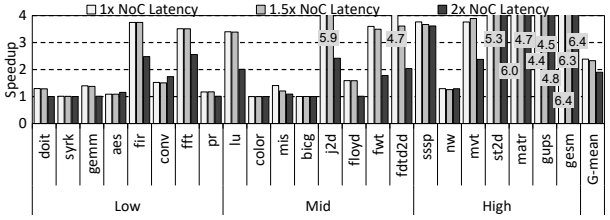


Figure 29: Performance under diverse NoC latency.

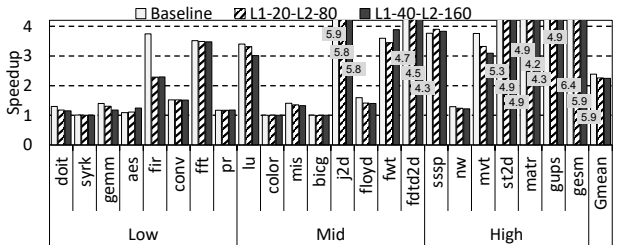


Figure 30: Performance under diverse TLB latency.

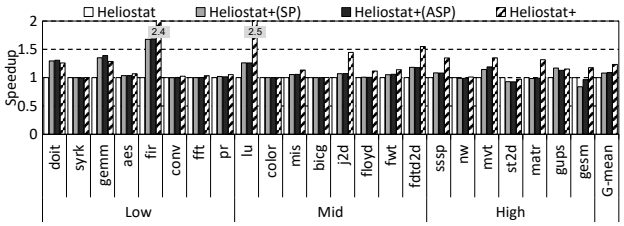


Figure 31: Performance with diverse TLB prefetchers

of diverse TLB latency. Some recent studies used longer TLB latencies (20-cycle L1 TLB and 80-cycle L2 TLB) [21]. We tested with this new setting and with even twice longer latencies as a stress test. As shown in Figure 30, the longer TLB latencies marginally drop the speedup from 2.39× to 2.26× and 2.25×, respectively. This is because translation throughput determines the overall performance in GPU rather than individual translation latencies.

8.7.6 TLB prefetcher. While Heliostat+ introduces an RTA-enabled lookahead translation mechanism, Heliostat+ primarily provides a framework for lookahead translation and can be integrated with any prefetching algorithms. Figure 31 presents the performance of Heliostat+ when the lookahead algorithm is replaced with other TLB prefetchers, sequential prefetcher (SP) and table-based arbitrary stride prefetcher (ASP), which are well evaluated in CPU domain [51]. With SP and ASP, Heliostat+ achieves 8.2% and 9.1% speedup over the baseline Heliostat. However, Heliostat+ outperforms when integrated with our proposed lookahead mechanism because we detect strides within each similar VPN region, unlike CPU TLB prefetchers which detect strides per load instruction. We observed that ASP's stride detection per load instruction leads to extremely diverse stride estimation because each load instruction is executed by hundreds of threads concurrently in GPU. Also, SP's next page translation supports only a certain access pattern for a

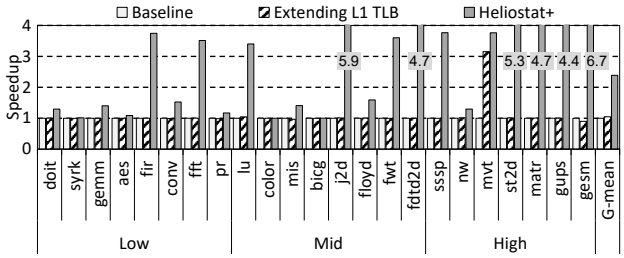


Figure 32: Performance comparison between Heliostat+ and repurposed L1S as victim cache of L1 TLB

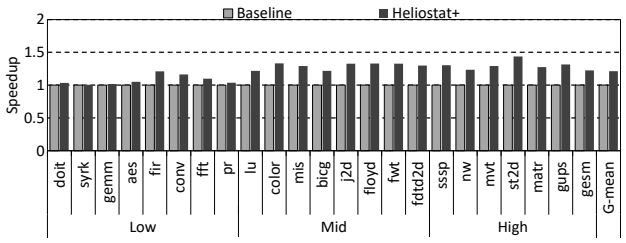


Figure 33: Speedup under memory oversubscription

certain page mapping (e.g., linear access when consecutive pages are mapped on the same CU), and hence cannot support broader GPU workloads.

8.8 L1 TLB extension with L1S

Heliostat+ utilizes L1S cache as a page walk cache. To understand the impact of this extra cache space, we compare the performance of Heliostat+ with another scenario where the same area of the L1S cache is repurposed as a victim cache for the L1 TLB, similar to a previous work [19]. As shown in Figure 32, most applications except for mvt show negligible speedup with the extended L1 TLB. This reaffirms the limitations of enlarged TLBs, as discussed in Section 3. With the same amount of extra cache space, Heliostat+ fundamentally alleviates the parallelism bottleneck of page walks.

8.9 Benefits under Memory Oversubscription

We evaluate Heliostat+ under GPU memory oversubscription (150% of GPU memory), with considering TLB flushing, CU pipeline draining, and page migration overhead. Using ACUD [4] for CU flushing and a counter-based migration scheme [12, 22, 48, 53], we find Heliostat+ maintains 1.21× speedup despite page fault handling typically being the primary bottleneck during active CPU-GPU page migration [4, 12, 18, 22, 30, 48, 53]. This sustained benefit stems from improved translation throughput for resident pages and increased PTW bandwidth enabling faster page fault handling.

8.10 Hardware Overhead and Energy Efficiency

We implemented PDU+ in Verilog and synthesized using Synopsys Design Compiler and FreePDK45. Adding four PDU+ to each of 128 RTAs incurs 0.024 mm^2 area (6.14%) and 4.484 mW power (39.02%) overhead compared to baseline GMMU with 16 PTWs and 16-entry single-ported CAM. Compared to a 128-PTW GMMU (which achieves similar speedup), PDU+ requires only 1.53% area

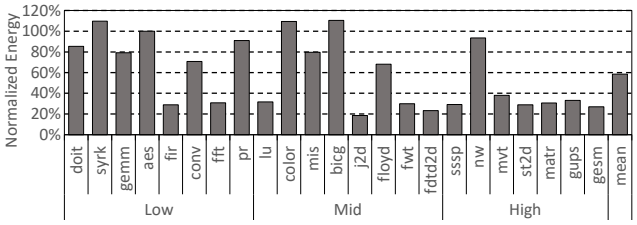


Figure 34: Normalized energy consumption of address translations of Heliostat+ over baseline.

and 5.8% power overhead. We also evaluated the energy consumption of page translation by accounting for the leakage and dynamic power of GMMU, PTW, PTW Cache, PDU, and L1S over the entire execution time. As shown in Figure 34, although Heliostat+ consumes slightly more energy for some applications (e.g., `syrk` and `color`), as Heliostat+ has significant speedup over baseline and uses existing L1S, the energy consumption is reduced by 41.42% on average. RPF implementation uses 32 cuckoo filters, each with 16 rows and 4-way associativity (2048 8-bit fingerprints total, using 4 packed bits with 3-bit compression and 5 direct bits)[11], achieving 1.53% theoretical false positive rate. The 2KB total filter size represents 1.6% area overhead versus baseline L2 TLB per CACTI[31] measurements.

9 Related Works

9.1 GPU Virtual Memory

Virtual memory on GPU is intensively studied. Power et al. [39] discussed the x86-style virtual memory on GPUs and exploration of design spaces. Ausavarungnirun et al. [3, 5] investigated translation versus paging overhead trade-offs in GPU. SnakeByte [21] proposed opportunistic page coalescing into variable sizes upon detecting mapping contiguity. Li et al. [22, 24, 25] explored Multi-GPU optimizations through TLB sharing, GMMU bypassing, and PTE lazy invalidation. Feng et al. [12] addressed Multi-Chip GPU translation through structural memory allocation and allocation hints for efficient coalescing. **However, existing works overlook the unmatched concurrency between GMMU and GPU CUs' memory execution, and heavily rely on opportunistic physical memory contiguity. Heliostat provides extra page translation parallelism through RTAs while requiring no page allocation contiguity.**

9.2 Democratizing RTA

Recent research has explored RTAs' massive memory parallelism beyond graphics workloads. RTNN [59] reformulated spatial neighbor search as a ray tracing problem. Nagarajan et al.[28, 32] extended it to non-Euclidean distances and unrestricted searching. Liu et al.[26] adapted RTA-based neighbor search for product quantization, while Lv et al.[27] and Wang et al.[54] applied RTAs to database scanning and outlier detection. TTA [14] and HSU [7] modified RTAs to traverse general-purpose tree structures beyond BVH. **To the best of our knowledge, Heliostat is the first study to extend RTA capabilities to general-purpose system operations, specifically virtual memory page table walks. We demonstrate fundamental similarities between RTA's BVH traversal and radix-tree-based page table walks. Heliostat introduces a novel approach to offload page table walks to RTAs, complementing existing solutions while showing superior performance compared to two state-of-the-art alternatives [8, 12].**

10 Conclusion

This paper tackles the limited translation bandwidth by leveraging the underutilized ray tracing accelerators without costly GMMU overprovisioning. By exploiting the operational similarities between page table walk and ray tracing, we propose to leverage RTAs for page table walk and further leverage the existing ray functions for lookahead translation. With significantly lower hardware overhead compared to the GMMU overprovisioning, Heliostat achieves 2.39× speedup over the baseline.

Acknowledgments

This work was supported by NSF grants, CCF-2114514 and CAREER-2341039. Part of this research was conducted using Pinnacles (NSF MRI, # 2019144) at the Cyber Infrastructure and Research Technologies (CIRT) at the University of California Merced, and Ampere® Altra® processors in servers donated by Ampere Computing.

References

[1] AMD. 2025. OminiPerf Documentation. https://rocm.docs.amd.com/projects/omniperf/en/latest/conceptual/shader-engine.html#desc-sl1d

[2] Nadav Amit, Amy Tai, and Michael Wei. 2020. Don't shoot down TLB shootdowns!. In *Proceedings of the Fifteenth European Conference on Computer Systems* (Heraklion, Greece) *(EuroSys '20)*. Association for Computing Machinery, New York, NY, USA, Article 35, 14 pages. doi:10.1145/3342195.3387518

[3] Rachata Ausavarungnirun, Joshua Landgraf, Vance Miller, Saugata Ghose, Jayneel Gandhi, Christopher J. Rossbach, and Onur Mutlu. 2017. Mosaic: a GPU memory manager with application-transparent support for multiple page sizes. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture* (Cambridge, Massachusetts) *(MICRO-50 '17)*. Association for Computing Machinery, New York, NY, USA, 136–150. doi:10.1145/3123939.3123975

[4] Rachata Ausavarungnirun, Joshua Landgraf, Vance Miller, Saugata Ghose, Jayneel Gandhi, Christopher J. Rossbach, and Onur Mutlu. 2017. Mosaic: a GPU memory manager with application-transparent support for multiple page sizes. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture* (Cambridge, Massachusetts) *(MICRO-50 '17)*. Association for Computing Machinery, New York, NY, USA, 136–150. doi:10.1145/3123939.3123975

[5] Rachata Ausavarungnirun, Vance Miller, Joshua Landgraf, Saugata Ghose, Jayneel Gandhi, Adwait Jog, Christopher J Rossbach, and Onur Mutlu. 2018. Mask: Redesigning the gpu memory hierarchy to support multi-application concurrency. *ACM SIGPLAN Notices* 53, 2 (2018), 503–518.

[6] Yuhui Bao, Yifan Sun, Zlatan Feric, Michael Tian Shen, Micah Weston, José L. Abellán, Trinayan Baruah, John Kim, Ajay Joshi, and David Kaeli. 2023. NaviSim: A Highly Accurate GPU Simulator for AMD RDNA GPUs. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques* (Chicago, Illinois) *(PACT '22)*. Association for Computing Machinery, New York, NY, USA, 333–345. doi:10.1145/3559009.3569666

[7] Aaron Barnes, Fangjia Shen, and Timothy G. Rogers. 2024. Extending GPU Ray-Tracing Units for Hierarchical Search Acceleration . In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, Los Alamitos, CA, USA, 1027–1040. doi:10.1109/MICRO61859.2024.00079

[8] Trinayan Baruah, Yifan Sun, Saiful A. Mojumder, José L. Abellán, Yash Ukidave, Ajay Joshi, Norman Rubin, John Kim, and David Kaeli. 2020. Valkyrie: Leveraging Inter-TLB Locality to Enhance GPU Performance. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques* (Virtual Event, GA, USA) *(PACT '20)*. Association for Computing Machinery, New York, NY, USA, 455–466. doi:10.1145/3410463.3414639

[9] Jichuan Chang and Gurindar S. Sohi. 2007. Cooperative cache partitioning for chip multiprocessors. In *ACM International Conference on Supercomputing 25th Anniversary Volume* (Munich, Germany). Association for Computing Machinery, New York, NY, USA, 402–412. doi:10.1145/2591635.2667188

[10] Nosayba El-Sayed, Anurag Mukkara, Po-An Tsai, Harshad Kasture, Xiaosong Ma, and Daniel Sanchez. 2018. KPart: A Hybrid Cache Partitioning-Sharing Technique for Commodity Multicores . In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 104–117. doi:10.1109/HPCA.2018.00019

[11] Bin Fan, Dave G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. 2014. Cuckoo Filter: Practically Better Than Bloom. In *Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies* (Sydney, Australia) *(CoNEXT '14)*. Association for Computing Machinery, New York, NY, USA, 75–88. doi:10.1145/2674005.2674994

[12] Yuan Feng, Seonjin Na, Hyesoon Kim, and Hyeran Jeon. 2024. Barre Chord: Efficient Virtual Memory Translation for Multi-Chip-Module GPUs . In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 834–847. doi:10.1109/ISCA59077.2024.00065

[13] Krishnan Gosakan, Jaehyun Han, William Kuszmaul, Ibrahim N. Mubarek, Nirjhar Mukherjee, Karthik Sriram, Guido Tagliavini, Evan West, Michael A. Bender, Abhishek Bhattacharjee, Alex Conway, Martin Farach-Colton, Jayneel Gandhi, Rob Johnson, Sudarsun Kannan, and Donald E. Porter. 2023. Mosaic Pages: Big TLB Reach with Small Pages. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (<conf-loc>, <city>Vancouver</city>, <state>BC</state>, <country>Canada</country>, </conf-loc>) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 433–448. doi:10.1145/3582016.3582021

[14] Dongho Ha, Lufei Liu, Yuan Hsi Chou, Seokjin Go, Won Woo Ro, Hung-Wei Tseng, and Tor M. Aamodt. 2024. Generalizing Ray Tracing Accelerators for Tree Traversals on GPUs . In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, Los Alamitos, CA, USA, 1041–1057. doi:10.1109/MICRO61859.2024.00080

[15] Intel. 2024. Arc A-series GPU architecture whitepaper. https://cdrdv2-public.intel.com/758302/introduction-to-the-xe-hpg-architecture-white-paper.pdf

[16] Aamer Jaleel, Eiman Ebrahimi, and Sam Duncan. 2019. Ducati: High-performance address translation by extending tlb reach of gpu-accelerated systems. *ACM Transactions on Architecture and Code Optimization (TACO)* 16, 1 (2019), 1–24.

[17] Norman P Jouppi. 1990. Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers. *ACM SIGARCH Computer Architecture News* 18, 2SI (1990), 364–373.

[18] Hyojong Kim, Jaewoong Sim, Prasun Gera, Ramyad Hadidi, and Hyesoon Kim. 2020. Batch-Aware Unified Memory Management in GPUs for Irregular Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 1357–1370. doi:10.1145/3373376.3378529

[19] Jagadish B. Kotra, Michael LeBeane, Mahmut T. Kandemir, and Gabriel H. Loh. 2021. Increasing GPU Translation Reach by Leveraging Under-Utilized On-Chip Resources. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 1169–1181. doi:10.1145/3466752.3480105

[20] Mohan Kumar Kumar, Steffen Maass, Sanidhya Kashyap, Ján Veselý, Zi Yan, Taesoo Kim, Abhishek Bhattacharjee, and Tushar Krishna. 2018. LATR: Lazy Translation Coherence. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems* (Williamsburg, VA, USA) *(ASPLOS '18)*. Association for Computing Machinery, New York, NY, USA, 651–664. doi:10.1145/3173162.3173198

[21] Jiwon Lee, Ju Min Lee, Yunho Oh, William J. Song, and Won Woo Ro. 2023. SnakeByte: A TLB Design with Adaptive and Recursive Page Merging in GPUs . In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 1195–1207. doi:10.1109/HPCA56546.2023.10071063

[22] Bingyao Li, Yanan Guo, Yueqi Wang, Aamer Jaleel, Jun Yang, and Xulong Tang. 2023. IDYLL: Enhancing Page Translation in Multi-GPUs via Light Weight PTE Invalidations. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (<conf-loc>, <city>Toronto</city>, <state>ON</state>, <country>Canada</country>, </conf-loc>) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 1163–1177. doi:10.1145/3613424.3614269

[23] Bingyao Li, Yueqi Wang, Tianyu Wang, Lieven Eeckhout, Jun Yang, Aamer Jaleel, and Xulong Tang. 2024. STAR: Sub-Entry Sharing-Aware TLB for Multi-Instance GPU . In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, Los Alamitos, CA, USA, 309–323. doi:10.1109/MICRO61859.2024.00031

[24] Bingyao Li, Jieming Yin, Anup Holey, Youtao Zhang, Jun Yang, and Xulong Tang. 2023. Trans-FW: Short Circuiting Page Table Walk in Multi-GPU Systems via Remote Forwarding . In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 456–470. doi:10.1109/HPCA56546.2023.10071054

[25] Bingyao Li, Jieming Yin, Youtao Zhang, and Xulong Tang. 2021. Improving Address Translation in Multi-GPUs via Sharing and Spilling Aware TLB Design. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 1154–1168. doi:10.1145/3466752.3480083

[26] Zihan Liu, Wentao Ni, Jingwen Leng, Yu Feng, Cong Guo, Quan Chen, Chao Li, Minyi Guo, and Yuhao Zhu. 2024. JUNO: Optimizing High-Dimensional Approximate Nearest Neighbour Search with Sparsity-Aware Algorithm and Ray-Tracing Core Mapping. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 549–565. doi:10.1145/3620665.3640360

[27] Yangming Lv, Kai Zhang, Ziming Wang, Xiaodong Zhang, Rubao Lee, Zhenying He, Yinan Jing, and X Sean Wang. 2024. RTScan: Efficient Scan with Ray Tracing Cores. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1460–1472.

[28] Durga Keerthi Mandarapu, Vani Nagarajan, Artem Pelenitsyn, and Milind Kulkarni. 2024. Arkade: k-Nearest Neighbor Search With Non-Euclidean Distances using GPU Ray Tracing. In *Proceedings of the 38th ACM International Conference on Supercomputing* (Kyoto, Japan) *(ICS '24)*. Association for Computing Machinery, New York, NY, USA, 14–25. doi:10.1145/3650200.3656601

[29] Daniel Meister, Paritosh Kulkarni, Aaryaman Vasishta, and Takahiro Harada. 2024. HIPRT: A Ray Tracing Framework in HIP. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 7, 3 (2024), 1–18.

[30] Ugljesa Milic, Oreste Villa, Evgeny Bolotin, Akhil Arunkumar, Eiman Ebrahimi, Aamer Jaleel, Alex Ramirez, and David Nellans. 2017. Beyond the Socket: NUMA-Aware GPUs . In *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE Computer Society, Los Alamitos, CA, USA, 123–135. https://doi.ieeecomputersociety.org/

[31] Naveen Muralimanohar, Rajeev Balasubramonian, and Norman P Jouppi. 2009. CACTI 6.0: A tool to model large caches. *HP laboratories* 27 (2009), 28.

[32] Vani Nagarajan, Durga Mandarapu, and Milind Kulkarni. 2023. RT-kNNS Unbound: Using RT Cores to Accelerate Unrestricted Neighbor Search. In *Proceedings of the 37th ACM International Conference on Supercomputing* (Orlando, FL, USA) *(ICS '23)*. Association for Computing Machinery, New York, NY, USA, 289–300. doi:10.1145/3577193.3593738

[33] Kyle J. Nesbit and James E. Smith. 2004. Data Cache Prefetching Using a Global History Buffer. In *Proceedings of the 10th International Symposium on High Performance Computer Architecture (HPCA '04)*. IEEE Computer Society, USA, 96. doi:10.1109/HPCA.2004.10030

[34] NVIDIA. 2024. Hopper architecture whitepaper. https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper

[35] NVIDIA. 2024. Turing architecture whitepaper. https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf

[36] PCI-SIG. 2025. PCI-SIG 6.0 Specification. https://pcisig.com/pci-express-6.0-specification

[37] Binh Pham, Abhishek Bhattacharjee, Yasuko Eckert, and Gabriel H. Loh. 2014. Increasing TLB reach by exploiting clustering in page translations . In *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 558–567. doi:10.1109/HPCA.2014.6835964

[38] Binh Pham, Viswanathan Vaidyanathan, Aamer Jaleel, and Abhishek Bhattacharjee. 2012. CoLT: Coalesced Large-Reach TLBs. In *Proceedings of the 2012 45th Annual IEEE/ACM International Symposium on Microarchitecture* (Vancouver, B.C., CANADA) *(MICRO-45)*. IEEE Computer Society, USA, 258–269. doi:10.1109/MICRO.2012.32

[39] Jason Power, Mark D. Hill, and David A. Wood. 2014. Supporting x86-64 address translation for 100s of GPU lanes. In *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, Orlando, FL, USA, 568–578. doi:10.1109/HPCA.2014.6835965

[40] B. Pratheek, Neha Jawalkar, and Arkaprava Basu. 2021. Improving GPU Multi-tenancy with Page Walk Stealing. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, Seoul, Korea (South), 626–639. doi:10.1109/HPCA51647.2021.00059

[41] B Pratheek, Neha Jawalkar, and Arkaprava Basu. 2022. Designing Virtual Memory System of MCM GPUs. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, Chicago, IL, USA, 404–422. doi:10.1109/MICRO56248.2022.00036

[42] Timothy J Purcell, Ian Buck, William R Mark, and Pat Hanrahan. 2005. Ray tracing on programmable graphics hardware. In *ACM SIGGRAPH 2005 Courses*. ACM, ACM New York, NY, USA, 268–es.

[43] Mohammadreza Saed, Yuan Hsi Chou, Lufei Liu, Tyler Nowicki, and Tor M. Aamodt. 2022. Vulkan-Sim: A GPU Architecture Simulator for Ray Tracing. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, Chicago, IL, USA, 263–281. doi:10.1109/MICRO56248.2022.00027

[44] Daniel Sanchez and Christos Kozyrakis. 2011. Vantage: scalable and efficient fine-grain cache partitioning. *SIGARCH Comput. Archit. News* 39, 3 (June 2011), 57–68. doi:10.1145/2024723.2000073

[45] Carlo H Séquin and Eliot K Smyrl. 1989. Parameterized ray-tracing. *ACM SIGGRAPH Computer Graphics* 23, 3 (1989), 307–314.

[46] Seunghee Shin, Guilherme Cox, Mark Oskin, Gabriel H. Loh, Yan Solihin, Abhishek Bhattacharjee, and Arkaprava Basu. 2018. Scheduling Page Table Walks for Irregular GPU Applications. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, Los Angeles, CA, USA, 180–192. doi:10.1109/ISCA.2018.00025

[47] Seunghee Shin, Michael LeBeane, Yan Solihin, and Arkaprava Basu. 2018. Neighborhood-Aware Address Translation for Irregular GPU Applications. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, Fukuoka, Japan, 352–363. doi:10.1109/MICRO.2018.00036

[48] Yifan Sun, Trinayan Baruah, Saiful A. Mojumder, Shi Dong, Xiang Gong, Shane Treadway, Yuhui Bao, Spencer Hance, Carter McCardwell, Vincent Zhao, Harrison Barclay, Amir Kavyan Ziabari, Zhongliang Chen, Rafael Ubal, José L. Abellán, John Kim, Ajay Joshi, and David Kaeli. 2019. MGPUSim: Enabling Multi-GPU Performance Modeling and Optimization. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, Phoenix, AZ, USA, 197–209.

[49] Seunghwan Sung, Sujin Hur, Sungwoo Kim, Dongho Ha, Yunho Oh, and Won Woo Ro. 2023. MAD MAcce: Supporting Multiply-Add Operations for Democratizing Matrix-Multiplication Accelerators. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) *(MICRO '23)*. Association for Computing Machinery, New York, NY, USA, 367–379. doi:10.1145/3613424.3614247

[50] Xulong Tang, Ziyu Zhang, Weizheng Xu, Mahmut Taylan Kandemir, Rami Melhem, and Jun Yang. 2020. Enhancing Address Translations in Throughput Processors via Compression. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques* (Virtual Event, GA, USA) *(PACT '20)*. Association for Computing Machinery, New York, NY, USA, 191–204. doi:10.1145/3410463.3414633

[51] Georgios Vavouliotis, Lluc Alvarez, Vasileios Karakostas, Konstantinos Nikas, Nectarios Koziris, Daniel A. Jiménez, and Marc Casas. 2021. Exploiting page table locality for agile TLB prefetching. In *Proceedings of the 48th Annual International Symposium on Computer Architecture* (Virtual Event, Spain) *(ISCA '21)*. IEEE Press, Valencia, Spain, 85–98. doi:10.1109/ISCA52012.2021.00016

[52] Vulkan. 2025. https://www.vulkan.org/.

[53] Yueqi Wang, Bingyao Li, Aamer Jaleel, Jun Yang, and Xulong Tang. 2024. GRIT: Enhancing Multi-GPU Performance with Fine-Grained Dynamic Page Placement. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, Edinburgh, United Kingdom, 1080–1094. doi:10.1109/HPCA57654.2024.00085

[54] Ziming Wang, Kai Zhang, Yangming Lv, Yinglong Wang, Zhigang Zhao, Zhenying He, Yinan Jing, and X. Sean Wang. 2024. RTOD: Efficient Outlier Detection With Ray Tracing Cores. *IEEE Transactions on Knowledge and Data Engineering* 36, 12 (2024), 9192–9204. doi:10.1109/TKDE.2024.3453901

[55] Zi Yan, Daniel Lustig, David Nellans, and Abhishek Bhattacharjee. 2019. Nimble Page Management for Tiered Memory Systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) *(ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 331–345. doi:10.1145/3297858.3304024

[56] Ying Ye, Richard West, Zhuoqun Cheng, and Ye Li. 2014. COLORIS: A dynamic cache partitioning system using page coloring. In *2014 23rd International Conference on Parallel Architecture and Compilation Techniques (PACT)*. IEEE, Edmonton, AB, Canada, 381–392. doi:10.1145/2628071.2628104

[57] Xiangyao Yu, Christopher J. Hughes, Nadathur Satish, and Srinivas Devadas. 2015. IMP: indirect memory prefetcher. In *Proceedings of the 48th International Symposium on Microarchitecture* (Waikiki, Hawaii) *(MICRO-48)*. Association for Computing Machinery, New York, NY, USA, 178–190. doi:10.1145/2830772.2830807

[58] Yunan Zhang, Po-An Tsai, and Hung-Wei Tseng. 2022. SIMD2: a generalized matrix instruction set for accelerating tensor computation beyond GEMM. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 552–566. doi:10.1145/3470496.3527411

[59] Yuhao Zhu. 2022. RTNN: accelerating neighbor search using hardware ray tracing. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Seoul, Republic of Korea) *(PPoPP '22)*. Association for Computing Machinery, New York, NY, USA, 76–89. doi:10.1145/3503221.3508409

[60] Xiaotong Zhuang and S Lee Hsien-Hsin. 2006. Reducing cache pollution via dynamic data prefetch filtering. *IEEE Trans. Comput.* 56, 1 (2006), 18–31.